Reading Assignment

1. What are the advantages of Polymorphism?
2. How is Inheritance useful to achieve Polymorphism in Java?
3. What are the differences between Polymorphism and Inheritance in Java?

Answer:

Polymorphism:

- Enables flexible code that can work with different object types through a common interface
- Reduces code duplication and complexity through shared behavior
- Facilitates extension without modifying existing code
- Supports design patterns and modular architecture

Inheritance's Role in Polymorphism

- Creates "is-a" relationships that allow child classes to be used where parent classes are expected
- Enables method overriding, where subclasses can provide specialized implementations
- Forms class hierarchies that maintain common interfaces while allowing specialized behavior
- Provides the foundation for runtime polymorphism (dynamic method dispatch)

Differences Between Polymorphism and Inheritance

- Inheritance is a mechanism (parent-child relationship) while polymorphism is a principle (one interface, many implementations)
- Inheritance focuses on reusing code; polymorphism focuses on interface abstraction
- Inheritance establishes relationships; polymorphism leverages these relationships at runtime
- Inheritance is compile-time (static); polymorphism can be runtime (dynamic)
- Inheritance is implemented with "extends" or "implements"; polymorphism occurs when using parent references for child objects

Question: Alternatively, to compare items in the cart, instead of using the Comparator class I have mentioned, you can use the Comparable interface1 and override the compareTo()method. You can

refer to the Java docs to see the information of this interface.

- The Comparable interface should be implemented in Media
- Need natural ordering strategy

public int compareTo(Media other) { *// First compare by title* int titleComparison = this.getTitle().compareTo(other.getTitle()); *// If titles are equal, compare by cost* if (titleComparison == 0) {

return Double.compare(this.getCost(), other.getCost());

}

return titleComparison;

}

We can not have 2 ordering rules with Comparable. We should use Comparator instead if we want multiple ordering strategies.

To modify DVD with different ordering, we can overide compareTo method in DVD class to provide a different ordering rule:

```
public class DVD extends Media {
    private int length; // length in minutes
    @Override
    public int compareTo(Media other) {
        if (other instanceof DVD) {
            DVD otherDVD = (DVD) other;
            // First compare by title
```

```
            int titleComparison = this.getTitle().compareTo(otherDVD.getTitle());
            if (titleComparison != 0) {
                return titleComparison;
            }
            // Then by decreasing length (note the reversed comparison)
            int lengthComparison = Integer.compare(otherDVD.getLength(), this.getLength());
            if (lengthComparison != 0) {
                return lengthComparison;
            }
            // Finally by cost
            return Double.compare(this.getCost(), otherDVD.getCost());
        } else {
            // Handle comparison with other media types
            // You could either use the parent class implementation:
            return super.compareTo(other);
            // Or define how DVDs compare to other media types
        }
    }
    public int getLength() {
        return length;
    }
    public void setLength(int length) {
        this.length = length;
    }
}
```